Resource-Aware Federated Learning Client Selection

Random Selection vs FedAvg vs FedCompass vs Proposed Adaptive Selection

Group 3 | Advanced Algorithms | Reference: FedCompass, ICLR 2024

The problem: not every client is worth waiting for

Given N clients, choose a subset S^* each round to minimise

$$T_{training} + \lambda \cdot C_{communication}$$

while keeping model accuracy.

Why it is hard

- Clients differ in compute speed: a synchronous round waits for its straggler.
- Data is non-IID: dropping a slow client can drop a whole class.
- Every selected client costs two model transfers (down + up).
- Pleasing fast clients only starves the slow ones (fairness).



Our contribution lives in step 2 (and in how K is chosen).

Common testbed for all four arms

Task / model	MNIST, 2-layer CNN (paper's), Q = 200 local Adam steps (100 at 20 clients, paper Table 9), lr 0.003
Clients	5, 10, 20 simulated clients; also a 6-laptop live testbed over Wi-Fi
Data split	Class partition (Algo 4) and Dual Dirichlet (Algo 5), both non-IID
Speed regimes	homo (all equal), normal (σ = 30%), exp (exponential: heavy-tailed stragglers)
Budget	Equal simulated wall-clock: every arm gets the time FedAvg needs for 10 rounds
Seeds	42, 1, 2 for every config: 18 configs × 4 arms × 3 seeds = 216 runs

Six reported metrics

- Model accuracy (best / final)
- Simulated wall-clock time
- Communication cost (model transfers)
- Updates and seconds to 90% accuracy
- Participation fairness (Jain's index)
- Selection overhead (ms in scheduler)

Arm 1: Random Selection

BASELINE

How it works

Each round, pick a uniformly random subset of K = N/2 clients. Every selected client runs the same Q local steps; the server waits for the slowest, then aggregates.

- No profiling, no state: selection overhead is under 1 ms.
- Cheapest in communication: only K clients transfer per round.
- Represents 'treat all clients identically' from the brief.

Strengths

- Zero overhead, trivially fair in expectation
- Lowest communication per round

Weaknesses

- Round time set by whichever straggler got drawn
- Can miss classes for several rounds under non-IID
- Final accuracy swings by 10+ points between seeds

Implemented in selection.py (random_selection). Fairness ≈ 0.91 to 0.93 across configs because K/N draws are uneven over a short run.

Arm 2: FedAvg, all clients every round

BASELINE

How it works

Every client trains every round. Weighted aggregation by data share p_i, then a global evaluation. Runs a fixed 10 rounds, which also defines the time budget the other arms get.

- Verified against the paper: 10-seed baseline matches Table 15 (93.08% ± 4.16%) within tolerance.
- Perfectly fair by construction (Jain = 1.0).
- Round time = Q × slowest client's step time.

Strengths

- Sees all data every round: stable under non-IID
- Fairness 1.0, no scheduler to tune

Weaknesses

- Pays the full straggler cost every round
- 2N transfers per round: highest communication
- Under exp heterogeneity, idle time dominates (≈ 4,900 client-seconds at 10 clients)

Implemented in fedavg.py. This is Milestone 1 and the ground truth every other arm is measured against.

Arm 3: FedCompass (ICLR 2024)

REFERENCE

How it works

Semi-asynchronous: there are no rounds. The server profiles each client's speed and hands each one a different step count so clients of different speeds arrive together as an arrival group:

$$Q_i = \lfloor (T_{a(g)} - t_{now}) / speed_i \rfloor, \quad Q_{min} \le Q_i \le Q_{max}$$

- Fast clients do more steps instead of idling behind stragglers.
- Stale updates are down-weighted by a staleness function.
- Our port: deterministic simulated clock (no threads), decaying staleness exponent, p_i-weighted aggregation as in Algorithm 1.

Strengths

- Lowest idle time by far (112 s vs 1,459 s for FedAvg at 5 clients, exp)
- Best accuracy on 20-client class split (98.9%)

Weaknesses

- Fast clients dominate: fairness collapses to 0.49 to 0.72 under exp
- 20× more model transfers than FedAvg at 10 clients (3,915 vs 200)
- Performs up to 8× more local steps for the same wall-clock

Implemented in compass.py with an assert-based self-check of the scheduler that runs without torch.

Arm 4: Proposed Adaptive Selection

PROPOSED

$$\text{Score}_i = \alpha \cdot \text{speed}(S \cup i) + \beta \cdot \text{gain}(S \cup i) - \gamma \cdot C_i + \delta \cdot \text{age}_i$$

speed(S ∪ i)

T_fastest / max T_j over the chosen set. Set-aware: a candidate is scored by the round time it would force, because a synchronous round waits for its slowest member. T_i is an EMA of observed seconds per step; round 1 profiles everyone.

gain(S ∪ i)

Marginal class coverage: Σ_k √(class-k mass in S ∪ i) minus the same for S. Concave, so a second client holding the same digits is worth far less than the first. A small client with a rare digit wins.

C_i

Communication cost. Identical for every client here (same CNN both ways), so it cannot reorder a top-K. Kept because the brief names it; it bites only in the budget variant.

age_i

Rounds since last selected. Our addition: pure top-K by speed starves slow clients forever, wrecking fairness and discarding their data under non-IID.

Greedy over the set, not per client: scoring clients independently picked the biggest and fastest client every round and left whole digits out. α = 1, β = 1, γ = 0.1, δ = 0.5.

Two mechanisms on top of the score

1 Coverage-driven K

K is a cap, not a target.

- The greedy loop stops as soon as every class has at least cover_floor of its samples in the chosen set (default 0.5 / N).
- Under Dirichlet, a fixed K can be structurally unable to cover every digit: with 5 clients at seed 42, four digits each live on a single client, so K = 3 forgets a digit every round and accuracy oscillates by 30 points.
- The selector picks as many clients as the data needs, no more. This is our take on 'adapt K dynamically'.

2 Deadline steps

Each selected client gets its own step count.

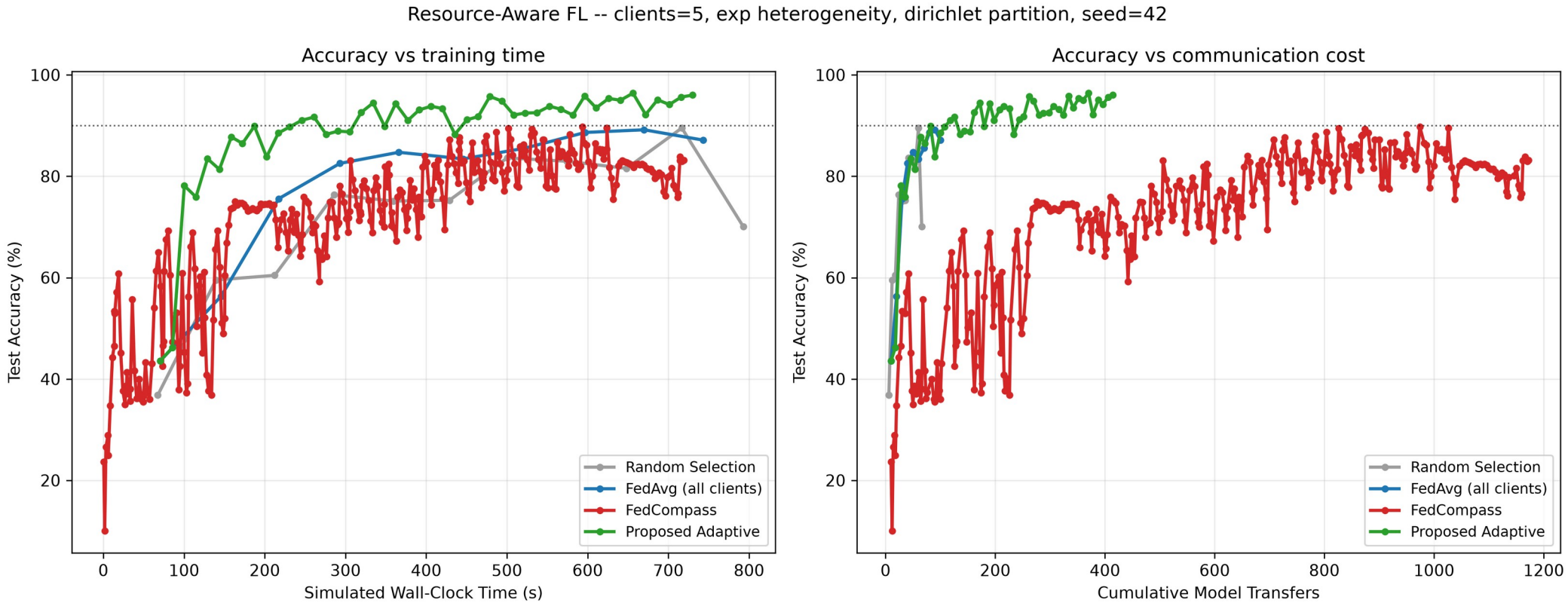
- $Q_i = \text{clamp}(Q \cdot T_{fastest} / T_i,\ Q/5,\ Q)$
- The round ends when the fastest selected client has done Q steps, not when the straggler has. A client 4× slower does 4× fewer steps and still arrives on time.
- This is FedCompass's idle-time fix, kept synchronous: one aggregation per round, no staleness, no arrival-group bookkeeping.

Both are switchable (--cover-floor 0, --no-deadline-steps), which is how the ablation on a later slide was run.

Results

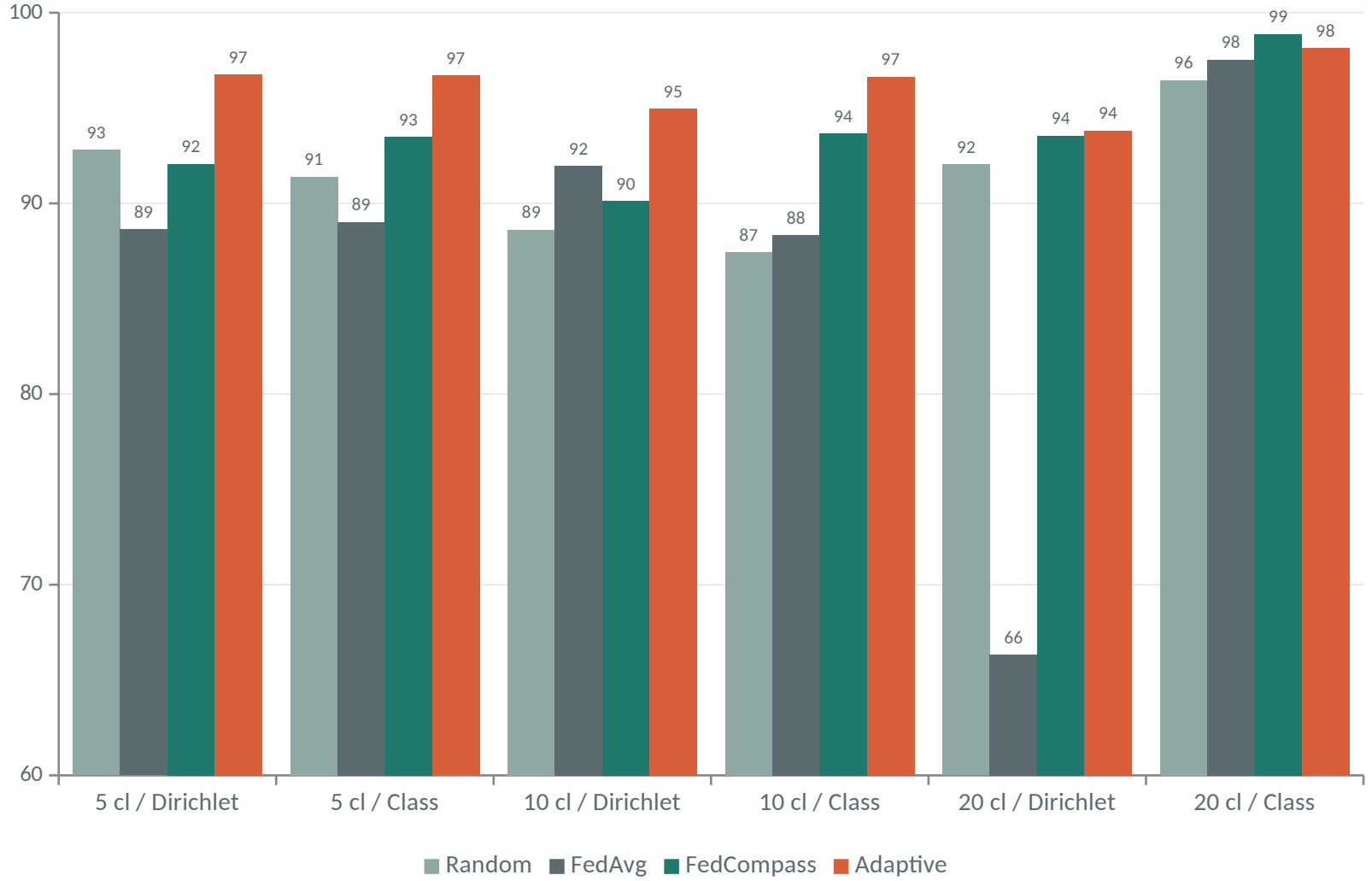
18 configurations × 3 seeds, all four arms on an equal simulated-time budget

In depth: 5 clients, exp speeds, Dirichlet, seed 42



Arm	Best acc	Final acc	To 90%	Transfers	Idle (s)	Fairness	Overhead
Random	89.5	70.1	never	66	1,018	0.95	0.6 ms
FedAvg	89.2	87.1	never	100	1,459	1.00	0.01 ms
FedCompass	89.8	83.2	never	1,176	112	0.58	8.9 ms
Adaptive	**96.4**	**96.0**	**246 s**	414	1,268	0.95	3.5 ms

Best accuracy under exp heterogeneity (mean of 3 seeds)

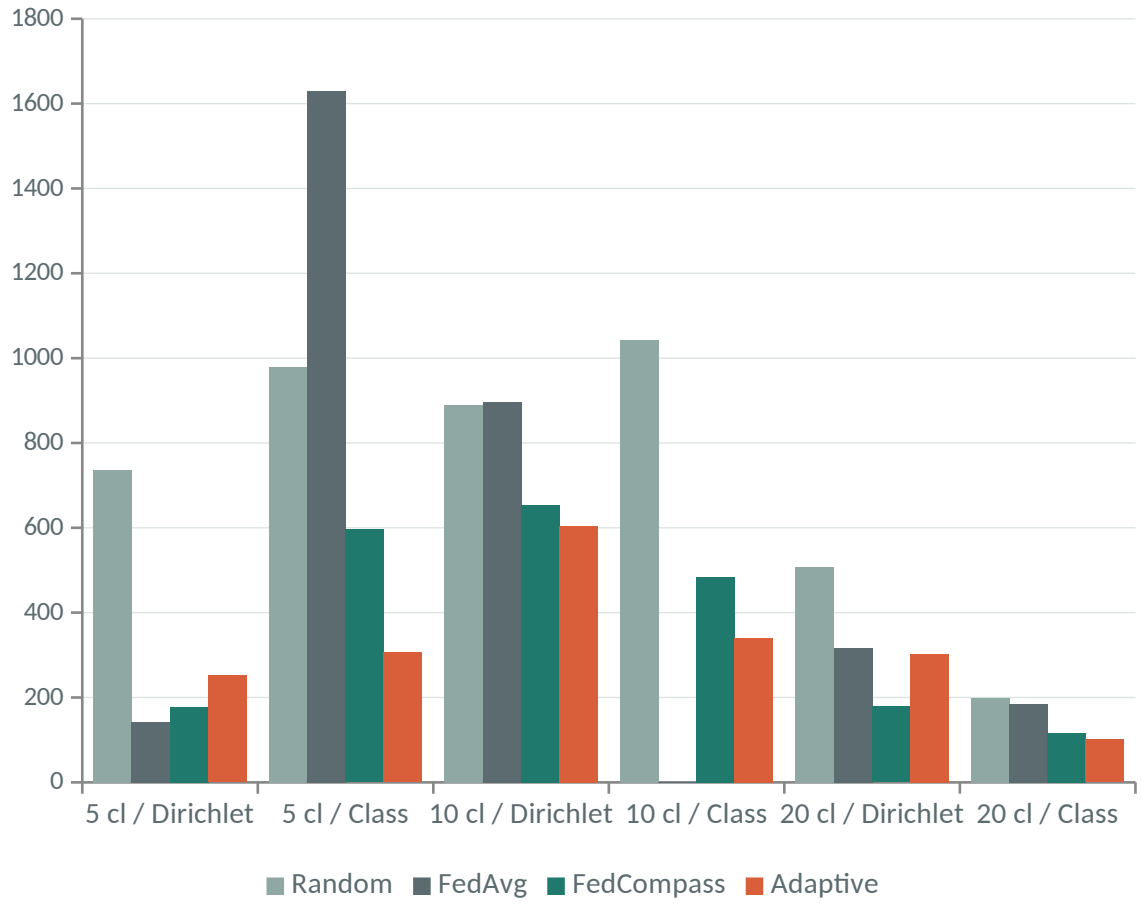


Adaptive leads in 5 of 6 cells

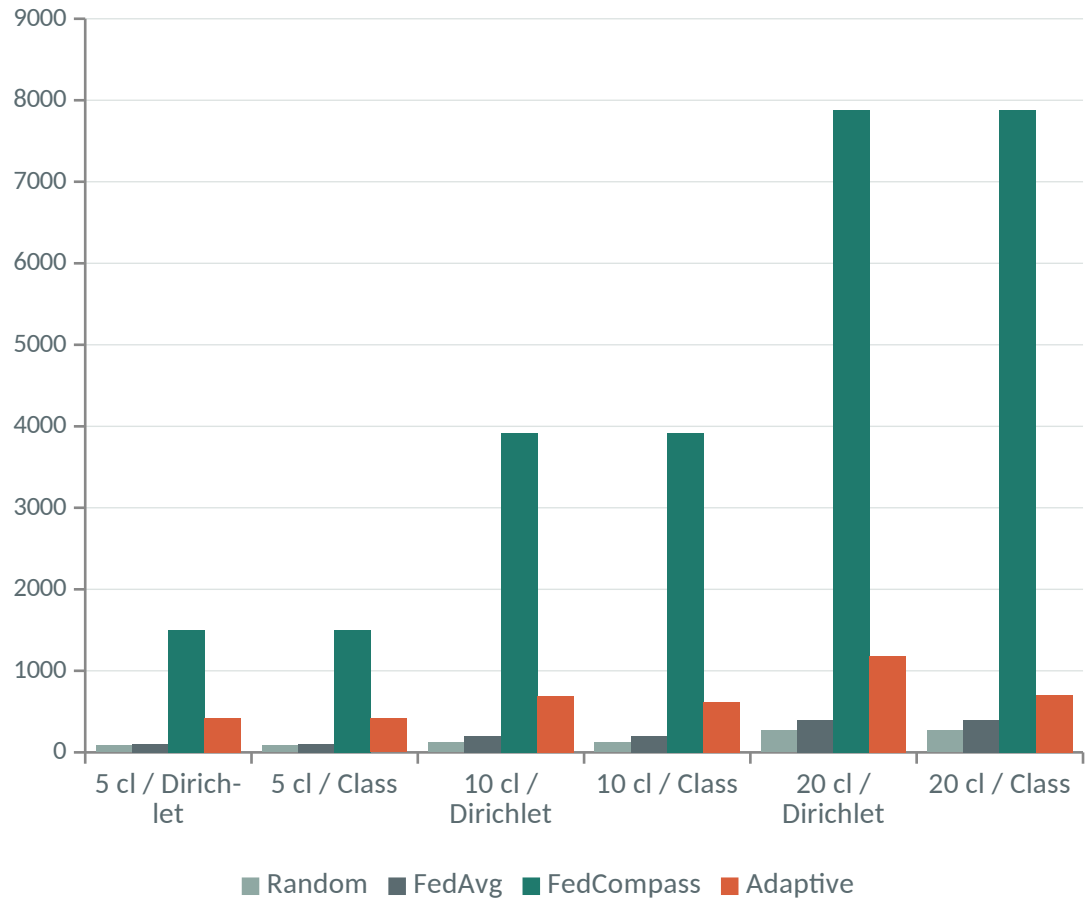
- +3 to +8 points over FedAvg at 5 and 10 clients.
- +3 to +5 points over FedCompass at 5 and 10 clients.
- 20 clients / Dirichlet: Adaptive edges FedCompass (93.8 vs 93.6); FedAvg diverged on one seed (std ≈ 39), its mean is not representative.
- 20 clients / Class: all arms above 96%, FedCompass best at 98.9%.

Speed and its price under exp heterogeneity

Simulated seconds to reach 90% (mean over seeds that reached it)



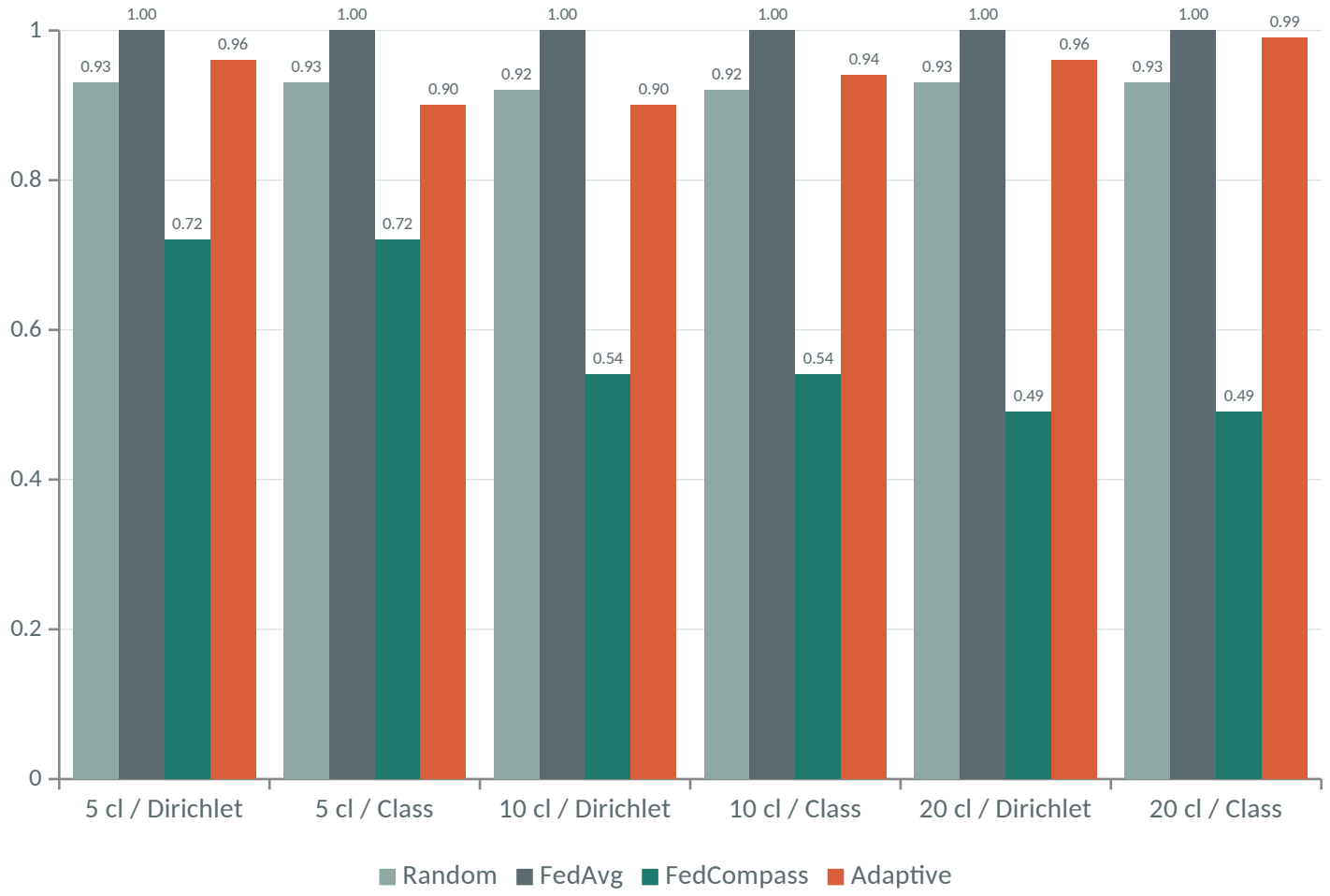
Model transfers over the same budget (down + up)



Adaptive reaches 90% fastest in 4 of 6 cells and is the only arm to hit 90% in every seed of every cell. FedAvg's 10-client Class bar is zero because it never reached 90%. FedCompass buys its speed with 15× to 20× FedAvg's transfers; Adaptive uses 2× to 4×.

Fairness and idle time: who gets to participate

Jain's fairness index over participation counts (1.0 = perfectly even), exp heterogeneity



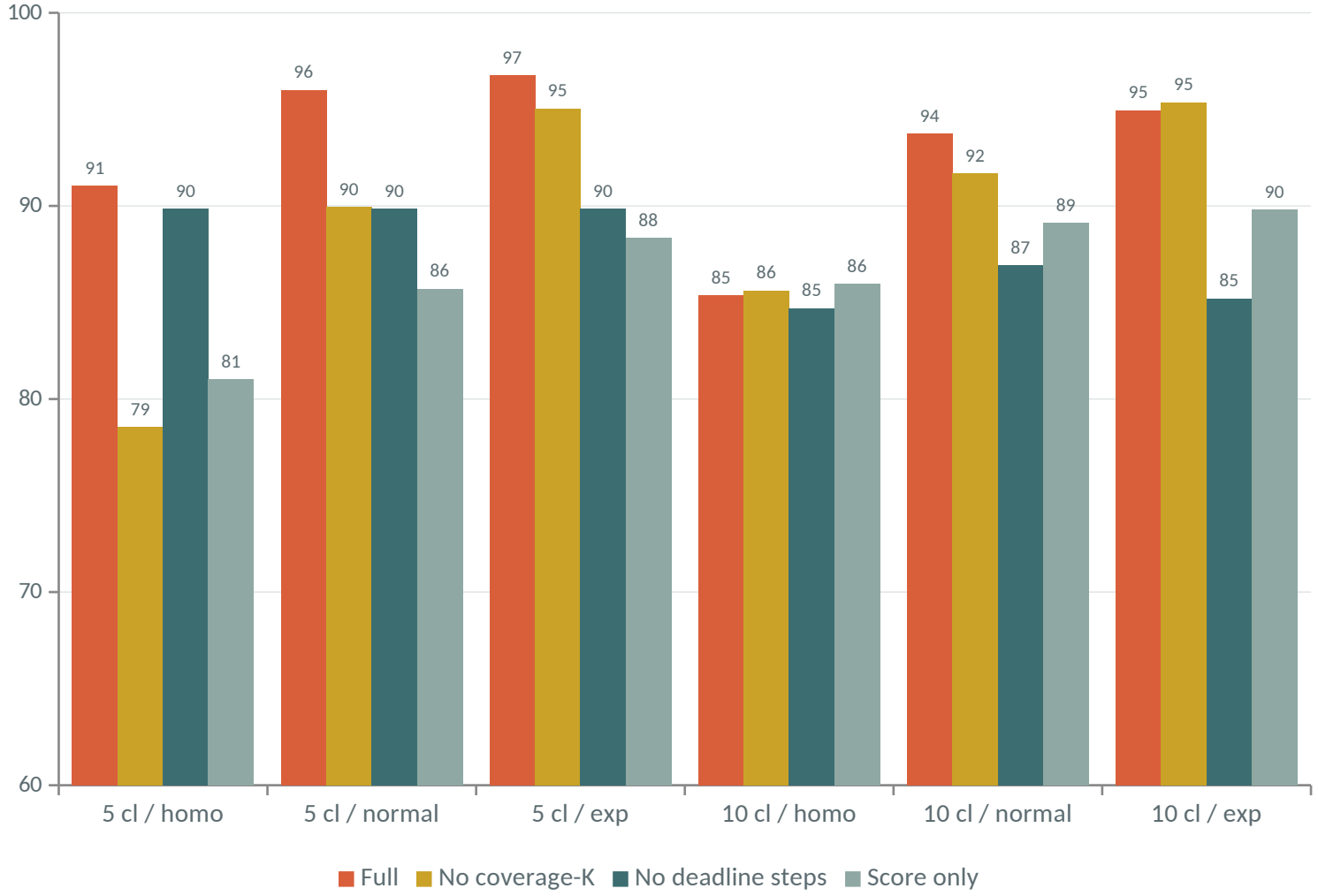
10 clients, exp, Dirichlet, seed 42

FedCompass participation per client:
24, 24, 24, 283, 383, 27, 30, 24, 383, 61
Three fast clients supply 83% of all updates. The global model drifts to their data and never reaches 90%.

Adaptive participation per client:
24, 24, 24, 47, 47, 24, 47, 47, 47, 47
Slow clients join every other round via the age term; nobody is starved.

Idle time is where FedCompass wins: 488 client-seconds vs 4,301 for Adaptive and 4,946 for FedAvg.

Ablation: which part of the adaptive arm does the work?



Best accuracy, Dirichlet split, mean of 3 seeds

- Deadline steps carry the heterogeneity gains: removing them costs 7 to 10 points under exp, and the no-deadline variant is identical across speed regimes (it just waits for the straggler).
- Coverage-driven K matters most when clients are few and classes are concentrated: −12 points at 5 clients / homo.
- The bare score is worst in 5 of 6 cells; the mechanisms, not the weights, are the contribution.
- Under homo at 10 clients the variants are within noise: with no stragglers there is nothing to schedule.

What the numbers do not say, and where we lose

Equal time ≠ equal compute

At equal simulated wall-clock, FedCompass performs about 1.6× (normal) to 8× (exp) more local steps than FedAvg, and Adaptive's deadline steps do the same in a smaller way. That is the mechanism, not an accounting trick, but 'reached 90% sooner' is a statement about elapsed time, not gradient steps.

20 clients / Dirichlet is unstable

Seed 1 collapses to chance for FedAvg under every speed model, and for Adaptive under homo and normal speeds (std ≈ 40 points). FedCompass's many small aggregations survive it; our once-per-round aggregation only does so under exp. Random and FedCompass are the honest winners of that cell.

Homogeneous clients: nothing to fix

With equal speeds there are no stragglers. FedAvg and FedCompass tie, and Adaptive's subset selection sometimes loses accuracy for the communication it saves (10 clients / homo / Dirichlet: 85.4 vs 92.0).

C_i is degenerate here

The same CNN ships both ways, so the communication term cannot reorder a top-K. It is kept because the brief names it and it bites once a budget variant grows K.

Takeaways

Random	Cheap and fair, but slow and noisy: the straggler you draw sets the round, and classes go missing.
FedAvg	The stable reference. Pays the full straggler cost every round; under exp heterogeneity most client time is idle.
FedCompass	Kills idle time and wins at scale, at the price of 15× to 20× the transfers and fairness down to 0.5: fast clients run the show.
Adaptive (ours)	Best accuracy and fastest to 90% at 5 and 10 clients, fairness ≈ 0.9+, transfers 2× to 4× FedAvg. A synchronous, lightweight middle ground.

Next: a budget-driven K that spends the C_i term; per-round micro-aggregation to close the 20-client Dirichlet gap; more seeds on the unstable cells.

Thank you

Group 3: Tirth Patel, Dhruven Gohel, Kirtan Rampariya, Punit Shah, Dhruve Patel, Krish Nariya

Code, 216 runs, ablation and the 6-laptop live testbed are in the repository.